REPOSITORY FIELD GUIDE · ANNOTATED COURSE

medgraphrag-journal

Hybrid Graph-RAG for Medical QA — the runner, manifest, and statistics harness behind three contributions: **RGD**, **CA-RRF**, and **CARe**.

244	118	$591	3
PLANNED RUNS	PASSING TESTS	BUDGET ENVELOPE	CONTRIBUTIONS

C1 · Retrieval-Generation Decomposition
C2 · Concept-Aware RRF
C3 · Cost-Aware Adaptive Reranking
Manifest-driven harness
Bootstrap · Permutation · Holm
RAGAS grounding
UMLS concept linking
RunPod / vLLM / NIM

A read-alongside walkthrough of the implementation, written to be consumed while browsing the source tree — not a README, but a guided course down to the nitty-gritty of every evaluation step.

Prepared for the thesis supervision dossier · generated 2026-06-27

0 What this repository is, and how to read this document

medgraphrag-journal is the experimental engine for a journal-grade upgrade of a Medical Graph-RAG study. It is not a model and not a single script: it is a **reproducible experimental program** — a frozen manifest of 244 runs, a deterministic runner, every retrieval arm from closed-book to concept-aware hybrid fusion, real grounding metrics, and a statistics layer engineered specifically to not repeat the validity mistakes of the original thesis.

The codebase is organized around one idea: **the experiment is data, the code is the engine that executes it.** A spreadsheet (`Experiment-Matrix.xlsx`) is the human contract for *what* to run; a frozen, hashed lock file is the machine contract the runner consumes; and every number that ever reaches the paper is traceable back through a per-item record to the exact configuration hash that produced it.

How to read this course

- **Follow the source tree alongside it.** Every section names the file it dissects (e.g. `mgr/stats/permutation.py`) so you can open the file and read the annotated excerpt in parallel.
- **Code excerpts are verbatim** copies of the repository, trimmed to the lines that matter. The prose around each block explains *why* the code is shaped the way it is — usually it is defending against a specific defect of the original thesis.
- **The evaluation is the spine.** Part 5 is the longest part on purpose: it walks every metric family, the answer-format audit, and the full statistics pipeline, line by line.
- **Three running symbols:** **C1** = Retrieval-Generation Decomposition, **C2** = Concept-Aware RRF, **C3** = Cost-Aware Adaptive Reranking. They reappear wherever a module implements part of that contribution.

Table of contents

PART I

Why this repository exists

A research harness is only as trustworthy as the mistakes it refuses to make. Almost every design choice here is a direct response to a concrete defect in the original thesis. Start here, because it explains the 'why' behind everything else.

1 The thesis defects and the journal upgrade

The original Medical Graph-RAG thesis produced an exciting headline — a hybrid graph system that topped retrieval metrics — but its evaluation contained several artifacts that a journal reviewer would (rightly) reject. The entire architecture of this repository is a set of **countermeasures**. The docstrings throughout the code refer back to these by name; this table is the master key.

Table 1. The defect→countermeasure ledger. Each row is a validity threat the harness neutralizes; the file column is where to read the fix.

#	Defect in the original study	Symptom	Countermeasure in this repo	Where
D-1	Arms scored under *different* normalized output schemas	EM jumps 0.00→0.76 for hybrids only, same generator	Single shared normalizer + a cross-arm **answer-format audit** that gates EM/F1	`eval/answer_format_audit.py`
D-2	Too few permutations in the significance test	Identical p = 0.00025 reported across three comparisons (the 1/N floor)	≥100k permutations; **refuses** to report the resolution floor as a point p	`stats/permutation.py`
D-3	Retrieval gains assumed to imply answer gains	Graph tops Recall@3/MRR but loses answer quality, unexplained	**RGD** — decompose & measure retrieval-gain vs generation-gain per system	`stats/rgd.py`
D-4	Concept fusion presented as a tunable black box	Reviewer can't isolate the concept signal's marginal value	**CA-RRF** with `use_concept=False` reproducing plain RRF exactly	`retrieval/ca_rrf.py`
D-5	Proxy 'grounding' metric	Faithfulness never actually measured against context	**Real RAGAS** (faithfulness, relevance, ctx precision/recall) via an injected judge	`metrics/grounding_ragas.py`
D-6	UMLS linking by exact match only (~47% coverage)	More than half of entity nodes never grounded	Exact + abbreviation + fuzzy linking, with a **measured coverage curve**	`graph/umls.py`
D-7	'Embedding named two ways' across a comparison	Arms silently differ in their embedding checkpoint	One frozen `embedding_checkpoint`; every record asserts parity	`configs/base.yaml`
D-8	No effect sizes; significance reported alone	Statistically 'significant' but practically tiny deltas	Paired Cohen's d + Cliff's delta reported beside every p	`stats/effect_size.py`

The through-line

Read the table top to bottom and a thesis emerges about the harness itself: **every headline number must be (a) computed under an audited, identical output schema, (b) accompanied by an honest confidence interval and an exact — never floored — p-value, (c) paired with an effect size, and (d) traceable to a frozen config hash.** The three named contributions (C1/C2/C3) sit on top of that foundation; the foundation is what makes them publishable.

2 Bird's-eye architecture & repository map

The system has four horizontal layers. A **contract & control** layer turns a spreadsheet into a frozen, hashed plan and a gate ledger. An **execution engine** resolves which rows are runnable, claims them, drives one deterministic run each, and writes immutable records. The **retrieval & generation arms** are interchangeable retrievers behind one protocol — the only thing that varies between experimental conditions. Finally a **measurement & evidence** layer scores items, audits them for comparability, runs the statistics, and emits the paper's figures and tables.

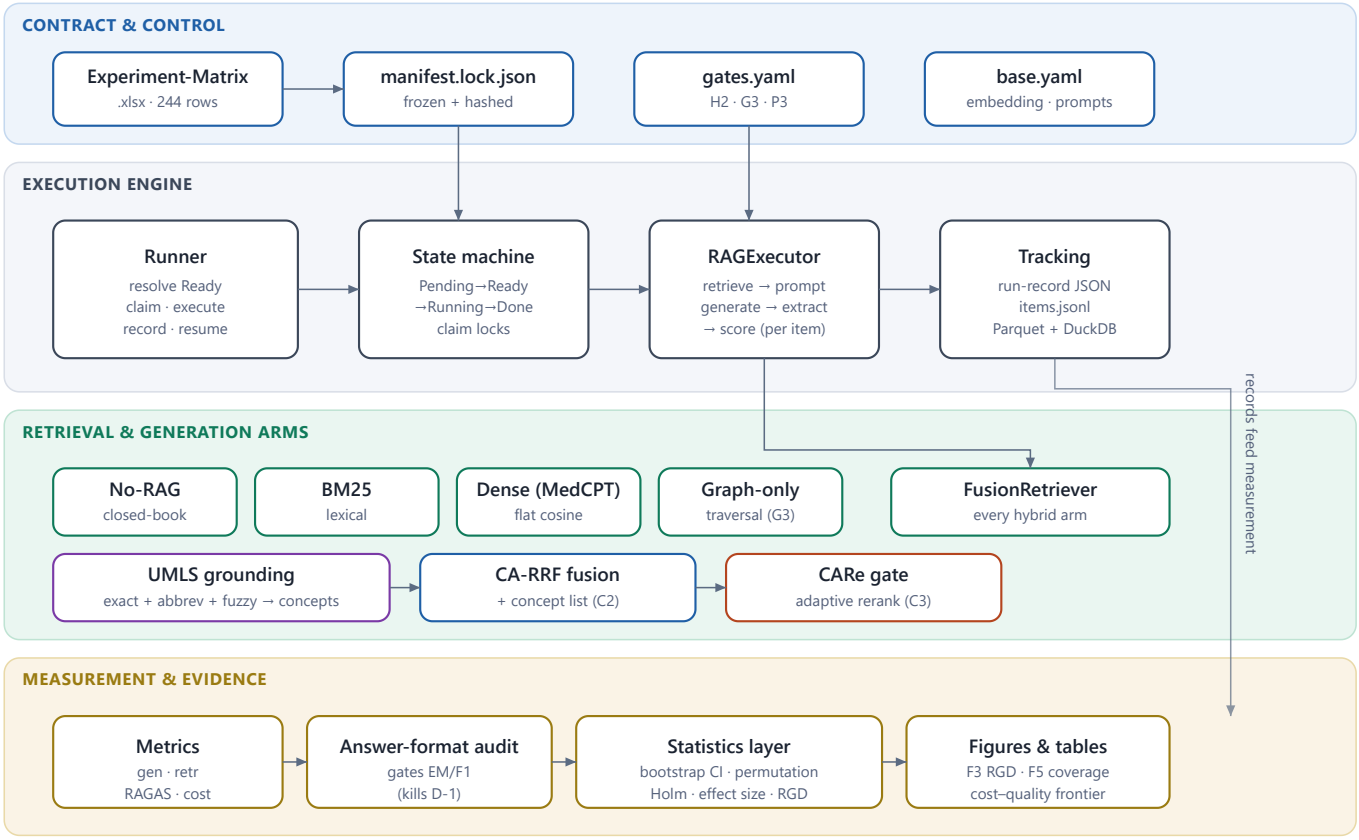

Figure 1. The four-layer architecture. Data flows top→bottom: the workbook is frozen to a lock file and gate ledger (blue); the runner executes one manifest row at a time through the RAGExecutor (grey); the condition selects which retrieval arm is wired in (green / C2 blue / C3 orange / grounding purple); per-item records feed the measurement layer (amber), where the answer-format audit gates EM/F1 before the statistics layer produces evidence. The condition is the *only* manipulated factor; prompt and scoring paths are shared across arms.

2.1 Reading the package layout

The repository splits cleanly into two installable packages — `manifest` (the contract) and `mgr` (the engine) — plus configuration, infra scripts, and tests. The table below is the map you will navigate for the rest of this document.

Table 2. The repository map. Two packages — `manifest` (the contract) and `mgr` (the engine) — organized by the four architectural layers.

Path	Layer	Responsibility
`manifest/manifest.py`	Contract	Load & validate the workbook; merge effective configs; reproduce the cost model
`manifest/lock.py`	Contract	Freeze the xlsx → `manifest.lock.json` (resolved params + hashes)
`configs/base.yaml`	Contract	The one embedding checkpoint, frozen prompts, decoding params, default depth
`configs/gates.yaml`	Contract	The H2/G3/P3 gate ledger that drives readiness
`mgr/ids.py`	Engine	run_id↔slug, config hashing, result-path resolution
`mgr/state.py`	Engine	Status enum, legal transitions, gate-driven readiness, claim locks
`mgr/runner.py`	Engine	One row → claim → execute → record; resume & integrity
`mgr/tracking/*`	Engine	Run-record schema; JSON + per-item JSONL + Parquet; DuckDB views
`mgr/clients/*`	Engine	Rate-limited OpenAI-compatible clients; NIM refuses generation; vLLM for 70B
`mgr/generate/*`	Arms	Frozen prompts, answer normalization, the generate→score executor
`mgr/retrieval/*`	Arms	Retriever protocol, BM25, dense, RRF, CA-RRF, FusionRetriever, factory
`mgr/rerank/*`	Arms	**C3** CARe gate, cross-encoder reranker, cost–quality frontier
`mgr/graph/umls.py`	Arms	**C2** UMLS concept linking with a coverage curve
`mgr/metrics/*`	Measurement	Generation, retrieval, cost, and real-RAGAS grounding metrics
`mgr/eval/answer_format_audit.py`	Measurement	Cross-arm schema/coverage audit that gates EM/F1
`mgr/stats/*`	Measurement	**C1** RGD, bootstrap CIs, permutation, Holm, effect sizes, qid alignment
`infra/*`	Lifecycle	RunPod up/serve/down, the with-pod trap, the idle-pod watchdog
`mgr/data/*`	Data	MIRAGE→JSONL converter; the benchmark loader
`tests/*`	Quality	118 offline tests across every module

The single most important invariant

Across every retrieval arm, **the prompt template, the answer normalizer, and the scoring code are byte-for-byte identical**. Only the injected retriever changes. This 'prompt parity' is what licenses the causal claim that a metric difference between two arms is caused by the retrieval condition and nothing else — it is the experimental-design backbone that the whole measurement layer rests on.

PART II

The experimental contract

Before a single token is spent, the entire experiment exists as data: a workbook of 244 rows, frozen into a hashed lock file, executed by a runner that can crash and resume without ever double-spending or silently corrupting a result. This part dissects that contract.

3.1 The workbook is the source of truth

The experiment lives in `manifest/Experiment-Matrix.xlsx`, a workbook with nine sheets. `Run_Manifest` is the spine: 244 rows, one per *(condition × benchmark × backbone × seed)* combination, keyed R0001–R0244. The other sheets are dimension tables (Conditions, Benchmarks, Backbones, Metrics), the statistical protocol, a live budget summary, and the phase gates.

Two contracts, one truth

The xlsx is the **human** contract — what to run, editable, reviewable. `manifest.lock.json` is the **machine** contract — every row resolved to its effective config plus a SHA-256 hash, frozen per sweep so that a mid-sweep edit to the workbook cannot silently change a running experiment. The runner reads the lock, never the xlsx.

`_sheet_records` reads any sheet into a list of header-keyed dicts, and `load_manifest` turns the rows into typed `RunRow` dataclasses. Note how dimension sheets are indexed by name so a row can later be joined to its full condition/benchmark/backbone definition:

manifest/manifest.py — sheets become name-indexed dimension tables

```
def _sheet_records(ws) -> list[dict[str, Any]]:
    """Return a worksheet's rows as dicts keyed by the header row."""
    rows = list(ws.iter_rows(values_only=True))
    if not rows:
        return []
    header = [str(h).strip() if h is not None else "" for h in rows[0]]
    out: list[dict[str, Any]] = []
    for r in rows[1:]:
        if all(c is None for c in r):
            continue
        out.append({header[i]: r[i] for i in range(len(header))})
    return out

# in load_manifest():
conditions = {r["condition"]: r for r in _sheet_records(wb["Conditions"]) ...}
benchmarks = {r["benchmark"]: r for r in _sheet_records(wb["Benchmarks"]) ...}
backbones  = {r["backbone"]:  r for r in _sheet_records(wb["Backbones"]) ...}
```

3.2 The design space: conditions, benchmarks, backbones

The power of the manifest is that the entire study is a Cartesian product over four dimensions, thinned by priority. The **twelve conditions** form a ladder from a parametric floor (No-RAG) up through single retrievers, the field-default fusions, the reproduced SOTA anchor, and the three contribution arms, with four ablations that each switch off exactly one mechanism.

Table 3. The twelve conditions (Conditions sheet). `ctx` is the context-length multiplier feeding the cost model. The two contribution arms are highlighted; the four `-no*/-static` ablations each isolate one mechanism by switching it off.

Condition	Retrievers	Fusion	Rerank	Grounding	ctx	Role
No-RAG	none (closed-book)	—	off	off	0.05	Parametric floor
BM25	BM25 lexical	—	off	off	0.9	Lexical ceiling
Dense-MedCPT	MedCPT	—	off	off	1.0	Dense ceiling
Graph-only	Graph traversal	—	off	on	0.7	Structural selection
MedGraphRAG-repro	Triple-graph + U-Retrieval	—	builtin	on	1.1	SOTA anchor
Hybrid-RRF2	BM25 + MedCPT	RRF-2	off	on	1.0	Fusion baseline
Hybrid-RRF4	BM25+Contriever+SPECTER+MedCPT	RRF-4	off	on	1.1	Strong baseline
Hybrid-CARRF	+ Graph + UMLS-concept list	CA-RRF	off	on	1.1	**C2** concept fusion
Hybrid-CARRF-staticRerank	+ Concept	CA-RRF	static	on	1.2	Rerank ablation
Hybrid-CARRF-CARe	+ Concept	CA-RRF	adaptive	on	1.15	**C3 — HEADLINE**
Hybrid-CARRF-noVecIndex	+ Concept (flat dense)	CA-RRF	off	on	1.1	Vector-index ablation
Hybrid-CARRF-noGrounding	+ Concept (no UMLS)	CA-RRF	off	off	1.0	Grounding ablation

The **benchmarks** span MCQ exams, yes/no/maybe reasoning, and long-form clinical QA; each declares its item count, a tokens-per-item estimate, and a metric focus that decides whether retrieval metrics even apply. The **backbones** are one primary 70B model for all main runs plus a three-model 'zoo' used to test whether the RGD phenomenon generalizes across model families.

Table 4. The benchmark suite (Benchmarks sheet). MIRAGE supplies the public MCQ/yes-no sets; GraphRAG-Bench and the governance-clean Clinical-Set stress long-form retrieval-heavy reasoning. `metric_focus` gates which metric families are reported per benchmark.

Benchmark	Source	Type	n_items	tok/item	Metric focus
MMLU-Med	MIRAGE	MCQ	1089	4500	Generation
MedQA-US	MIRAGE	MCQ	1273	5000	Generation
MedMCQA	MIRAGE	MCQ	4183	4500	Generation
PubMedQA	MIRAGE	yes/no/maybe	500	5500	Retrieval+Gen
BioASQ-YN	MIRAGE	yes/no	618	5000	Retrieval+Gen
GraphRAG-Bench-Med	External	long-form	50	8000	Retrieval+Gen
Clinical-Set	Own (synthetic/DUA)	long-form	200	8000	Retrieval+Gen
OverRefusal	Own	—	—	—	Safety / refusal

Table 5. The four backbones (Backbones sheet). The 70B model carries the 208 main runs; the 12-run 'zoo' per small model is a generality probe for **C1**. `$/M` is the blended per-million-token rate the cost model uses.

Backbone	Model id	Provider	Role	Ctx	$/M	Runs
Llama-70B	meta/llama-3.3-70b-instruct	NVIDIA NIM (hosted)	Primary	128k	0.60	208
Qwen-14B	Qwen2.5-14B-Instruct	Local vLLM	Zoo	32k	0.15	12
Mistral-24B	Mistral-Small-3 24B	Local vLLM	Zoo	32k	0.20	12
BioMed-8B	Biomedical Llama-3.1-8B	Local vLLM	Zoo	8k	0.10	12

3.3 Identity: run_id, slug, and config_hash

Every run has exactly one canonical key — a sequential `run_id` like `R0123` — and that key is **never** re-derived from the dimensions. A human-readable slug exists only for logs and filenames; it must round-trip its four components but is never primary. This separation prevents a whole class of bugs where two rows that differ in some hidden way collapse to the same 'name'.

`mgr/ids.py — the slug is cosmetic; the config_hash is the real fingerprint`

```
def slug(condition: str, benchmark: str, backbone: str, seed: int | str) -> str:
    """Derive the canonical human slug for a run."""
    return f"{condition}__{benchmark}__{backbone}__s{seed}"

def _canonical(obj):
    """Mappings key-sorted; sequences kept in order (list order is meaningful)."""
    if isinstance(obj, Mapping):
        return {k: _canonical(obj[k]) for k in sorted(obj)}
    if isinstance(obj, (list, tuple)):
        return [_canonical(v) for v in obj]
    return obj

def config_hash(config: Mapping[str, Any]) -> str:
    """sha256 of the merged, canonicalized effective config."""
    return hashlib.sha256(canonical_json(config).encode("utf-8")).hexdigest()
```

The hashing is deliberately asymmetric: dictionary key order is normalized away (so two configs that differ only in YAML key ordering hash identically), but **list order is preserved** because it is semantically meaningful — the order of retrievers in a fusion arm, for instance, changes the experiment. The result-path helpers (`record_path`, `items_path`, `claim_path`) all derive deterministically from the `run_id`, so a worker on any host computes the same paths.

3.4 Effective-config merge & the lock file

A row's **effective config** is a layered merge — `base (+) condition (+) benchmark (+) backbone (+) {seed}` — where later layers override earlier ones. That merged dict is exactly what gets canonicalized and hashed, so the `config_hash` captures every input that could affect the result.

`manifest/manifest.py — the merge that defines a run's identity`

```
def effective_config(self, run: RunRow) -> dict[str, Any]:
    """base (+) condition (+) benchmark (+) backbone (+) {seed}."""
    merged: dict[str, Any] = {"base": self.base}
    merged["condition"] = self.conditions[run.condition]
    merged["benchmark"] = self.benchmarks[run.benchmark]
    merged["backbone"]  = self.backbones[run.backbone]
    merged["seed"]      = run.seed
    merged["run_id"]    = run.run_id
    return merged
```

`manifest/lock.py` walks every row, computes its effective config and hash, and writes a lock document whose *own* top-level hash covers the entire frozen set. If the workbook changes, the lock hash changes, and any record produced under the old lock is detectably stale.

`manifest/lock.py — freezing the xlsx into a hashed machine contract`

```
def build_lock(m: Manifest) -> dict[str, Any]:
    entries = []
    for r in m.runs:
        cfg = m.effective_config(r)
        entries.append({
            "run_id": r.run_id, "slug": ids.slug(...),
            "condition": r.condition, "benchmark": r.benchmark,
            "backbone": r.backbone, "seed": r.seed,
            "depends_on_gate": r.gate,
            "est_tokens": r.computed_est_tokens(),
            "est_cost_usd": r.computed_est_cost(),
            "config_hash": ids.config_hash(cfg),
        })
    body = {"version": 1, "n_runs": len(entries), "runs": entries}
    body["manifest_lock_hash"] = _hash_body(body)   # hash-of-the-whole
    return body
```

What 'frozen' buys you

Running `python -m manifest.lock` first **validates** the workbook (next section) and only then writes the lock. The current frozen lock covers **244 runs** with `manifest_lock_hash = ab923780…`. Every run-record the runner writes embeds both its own `config_hash` and this `manifest_lock_hash`, so any result can be tied back to the precise plan that authorized it.

3.5 The cost model and budget reconciliation

A research budget that drifts from reality is a budget that gets blown. The manifest carries a cost model — `est_tokens = n_items × tokens_per_item × ctx_factor`, `est_cost = est_tokens / 1e6 × rate` — and `validate()` recomputes it for every row and asserts it matches the value the spreadsheet stored. A mismatch is a hard validation failure, so the workbook's formulas and the code can never silently diverge.

`manifest/manifest.py — four design invariants, checked before every freeze`

```
def validate(m: Manifest) -> list[str]:
    problems: list[str] = []
    # 1. run_id is sequential R0001.. with no gaps or dups
    # 2. every depends_on maps to a known gate key (H2/G3/P3)
    # 3. every condition/benchmark/backbone referenced exists in its sheet
    # 4. cost model reconciles with the sheet:
    for r in m.runs:
        if not math.isclose(r.computed_est_tokens(), r.est_tokens, rel_tol=1e-6, abs_tol=1.0):
            problems.append(f"{r.run_id}: est_tokens mismatch ...")
        if not math.isclose(r.computed_est_cost(), r.est_cost, rel_tol=1e-6, abs_tol=1e-6):
            problems.append(f"{r.run_id}: est_cost mismatch ...")
    return problems
```

The reconciled totals are the budget the whole program is held to. A separate cost meter (`mgr/metrics/cost.py`, covered in the evaluation part) tracks *actual* observed tokens against this estimate at run time, and because RunPod bills by GPU-hour rather than per token, the actual dollar cost lands well below the token-estimate ceiling when the A100 is saturated.

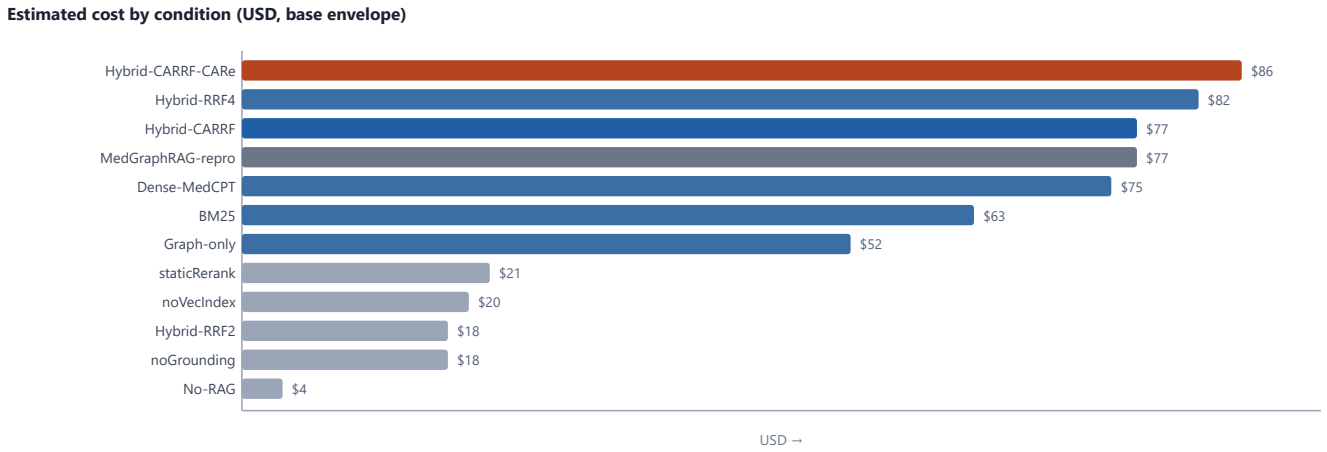

Figure 2. Estimated cost per condition. The headline C3 arm (orange) is the single most expensive condition because it carries the most runs (33, three seeds × many benchmarks); the four ablations (grey, 4 runs each) are deliberately cheap single-seed probes.

Table 6. Budget reconciliation (Budget_Summary sheet, reproduced exactly by the code). The base envelope is **$591.49**; a 1.5× rerun buffer gives the **$887.23** ceiling the program plans against. RAGAS judge tokens (~70M) are budgeted separately, post-hoc.

Aggregation	Runs	Est. tokens	Est. cost
Phase P2 (baselines + repro)	171	654,967,950	$352.48
Phase P3 (CA-RRF ablation)	40	255,151,250	$153.09
Phase P4 (CARe + zoo)	33	170,926,800	$85.92
TOTAL	**244**	**1,081,046,000**	**$591.49**
With 1.5× rerun buffer	—	1,621,569,000	$887.23
Llama-70B (primary)	208	954,073,250	$572.44
Zoo models (14B/24B/8B)	36	126,972,750	$19.05

3.6 Gates, the state machine & readiness

Not every run is runnable at every moment. A graph-touching arm cannot run until the grounded graph is built; the headline CARe arm cannot run until the CA-RRF results and oracle rerank labels exist. Rather than encode these dependencies as a brittle run-to-run DAG, the design uses a tiny **gate ledger** with exactly three keys. A row carries a `depends_on` gate; its readiness is a pure lookup against the ledger.

Table 7. The three-key gate ledger (`configs/gates.yaml`). Readiness is a ledger lookup, not a dependency graph: flipping one boolean to `satisfied: true` — backed by an auditable evidence artifact — unblocks an entire wave of rows.

Gate	Meaning	Set by	Unblocks	Rows
H2	Harness validated on the smoke set	Step 1 (No-RAG + BM25 smoke)	All baseline / main rows	118
G3	Grounded chunk-level graph frozen	Step 2 (graph build + UMLS)	Graph-only & every Hybrid-* arm	93
P3	CA-RRF results + oracle rerank labels	Step 5 (oracle labelling)	Hybrid-CARRF-CARe (the headline)	33

The state machine has six states with an explicitly enumerated transition table. `Done` is terminal; `Failed→Running` is permitted so a crashed run can resume. The crucial function is `resolve_status`, a pure function of (current status, gate key, ledger):

`mgr/state.py — readiness is a ledger lookup, nothing more`

```
def resolve_status(current, depends_on, gate_ledger) -> Status:
    current = Status(current)
    # Done / Running / Failed are owned by the runner and pass through.
    if current in (Status.DONE, Status.RUNNING, Status.FAILED):
        return current
    gate_key = gate_name(depends_on)            # "H2 (harness)" -> "H2"
    if gate_key not in gate_ledger:
        raise KeyError(f"unknown gate {gate_key!r}")
    return Status.READY if gate_ledger[gate_key] else Status.BLOCKED
```

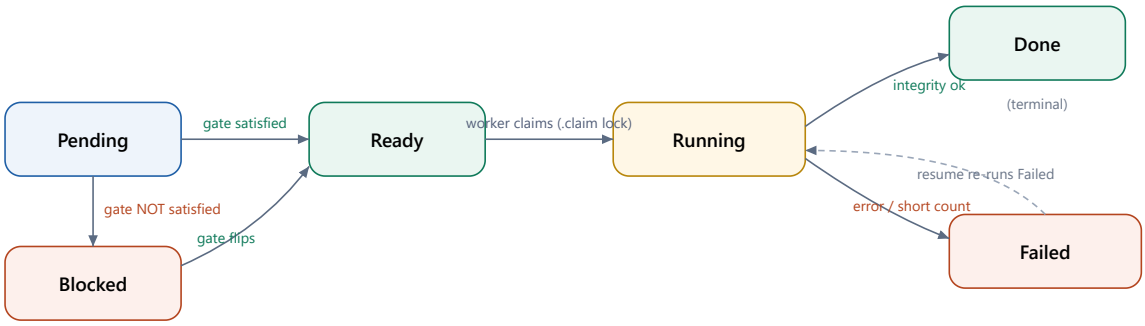

Figure 3. The run state machine. A row starts Pending; its gate decides Ready vs Blocked. A worker moves Ready→Running by writing an atomic `.claim` lock, then to Done only if the integrity check passes, else to Failed. Resume re-runs Failed rows (dashed); Done is terminal and is never re-executed.

3.7 The runner: claim → execute → record

The runner's unit of work is a single row. It never re-derives fan-out; it resolves Ready rows against the ledger, claims one with a filesystem lock, runs it, and writes an immutable record. Three guards make this safe under crashes and parallel workers:

- **Resume idempotence** — a present, integrity-passing `Done` record is never re-run, so a restarted sweep skips finished work for free.
- **Claim locks** — `acquire_claim` writes `results/per-run/{id}/.claim`; a live claim held by another worker means skip, and a stale claim (older than an hour) is reaped so a dead worker's row is reclaimed.
- **Integrity demotion** — a run that finishes but processed fewer items than intended is rewritten from Done to Failed, so a truncated data file can never masquerade as a complete result.

`mgr/runner.py — the crash-safe core loop (one row)`

```
def run_one(self, run_id, executor=stub_executor):
    row = self.manifest.by_id(run_id)
    # Resume: a present, integrity-passing Done record is never re-run.
    existing = rec.read_record(run_id, self.results_root)
    if existing is not None and rec.integrity_ok(existing, row.n_items):
        return None
    status = resolve_status(Status(row.status), row.depends_on, self.gate_ledger)
    if status != Status.READY:
        return None                              # Blocked: gate not satisfied
    claim = acquire_claim(run_id, self.results_root, self.stale_after_s)
    if claim is None:
        return None                              # held by a live worker
    try:
        cfg = self.manifest.effective_config(row)
        meter = CostMeter.from_estimate(row.n_items, row.tokens_per_item, row.ctx_factor, row.rate)
        result = executor(row, cfg)              # <-- the pluggable work
        ...
        # Done requires processing everything it intended to:
        expected = result.expected_n_items or result.n_items
        if record.status == "Done" and not rec.integrity_ok(record, expected):
            record.status = "Failed"
            record.error = f"integrity: processed {record.n_items} != expected {expected}"
        rec.log(record, self.results_root)
        return record
    finally:
        release_claim(run_id, self.results_root)
```

Pluggable execution

The runner is execution-agnostic: it takes an `Executor` callable mapping `(row, config) → ExecResult`. A `stub_executor` exercises the entire claim→record→release lifecycle *without spending a token* — it emits synthetic per-item rows so the downstream statistics join can be built and tested before any real generation exists. The real `RAGExecutor` (Part IV) slots into the same hole. This is why 118 tests can validate the full pipeline offline.

3.8 Tracking: records, JSONL, Parquet, DuckDB

The record of truth is plain files. Each run writes a `RunRecord` JSON (its config hash, lock hash, git SHA, embedding checkpoint, token counts, costs, and metrics) plus a per-item `items.jsonl` — the qid-keyed substrate the statistics layer later joins on. A roll-up step upserts all JSON records into a single Parquet table, and DuckDB reads that Parquet directly as queryable **views**. Crucially, dashboards and Parquet are *derived*; the JSON/JSONL files remain canonical even if pyarrow is absent.

`mgr/tracking/record.py — integrity gate + the derived Parquet roll-up`

```
def integrity_ok(record: RunRecord, expected_n_items: int) -> bool:
    """Done iff the run processed the full benchmark and emitted metrics."""
    return (
        record.status == "Done"
        and record.n_items == expected_n_items
        and bool(record.metrics)
        and record.error is None
    )

def roll_up_parquet(results_root="results"):
    # Flatten nested dicts (embedding_checkpoint, tokens, metrics) to JSON strings
    # for a stable columnar schema, then upsert into run_records.parquet.
    ...   # returns None if pyarrow unavailable -> JSON remains canonical truth
```

The DuckDB layer is zero-server: it opens the Parquet file as a view and answers status and cost queries that mirror the workbook's live budget summary. This is how the operator watches `cost_actual_usd` climb against the estimate during a sweep without standing up any infrastructure.

`mgr/tracking/store.py — DuckDB views are derived, never the truth`

```
def v_cost(parquet="results/run_records.parquet"):
    return query("""
        SELECT condition, backbone,
               SUM(cost_est_usd)    AS est_usd,
               SUM(cost_actual_usd) AS actual_usd
        FROM runs GROUP BY condition, backbone ORDER BY est_usd DESC
    """, parquet)
```

PART III

The three contributions

Everything so far is scaffolding so that these three ideas can be measured honestly. C1 names a phenomenon the field had hand-waved; C2 turns medical terminology into a first-class ranking signal; C3 spends reranking compute only where it pays off.

4.1 C1 · Retrieval-Generation Decomposition (RGD)

C1 is the conceptual headline. The thesis symptom was that graph systems *top retrieval* (Recall@3, MRR) while *losing answer quality* — a contradiction the field had previously waved away. RGD turns that contradiction into a measured object. For each system you compute two gains relative to a baseline, on the **same items**: a retrieval gain and a generation gain. A system whose retrieval gain is positive but whose generation gain is not is flagged as **divergent** — selection improved, but the improvement did not transfer to the answer.

`mgr/stats/rgd.py — the decomposition that names the phenomenon`

```
@dataclass(frozen=True)
class RGDPoint:
    system: str
    retrieval: float; generation: float
    retrieval_gain: float; generation_gain: float

    @property
    def diverges(self) -> bool:
        """Retrieval improves but generation does not (the C1 phenomenon)."""
        return self.retrieval_gain > 0.0 and self.generation_gain <= 0.0

def decompose(systems, baseline) -> list[RGDPoint]:
    br, bg = systems[baseline]                 # baseline retrieval, generation
    points = [RGDPoint(name, r, g, r - br, g - bg) for name, (r, g) in systems.items()]
    points.sort(key=lambda p: p.retrieval_gain, reverse=True)
    return points

def divergent_systems(points) -> list[str]:
    return [p.system for p in points if p.diverges]
```

RGD plane: retrieval gain (x) vs generation gain (y), vs a baseline

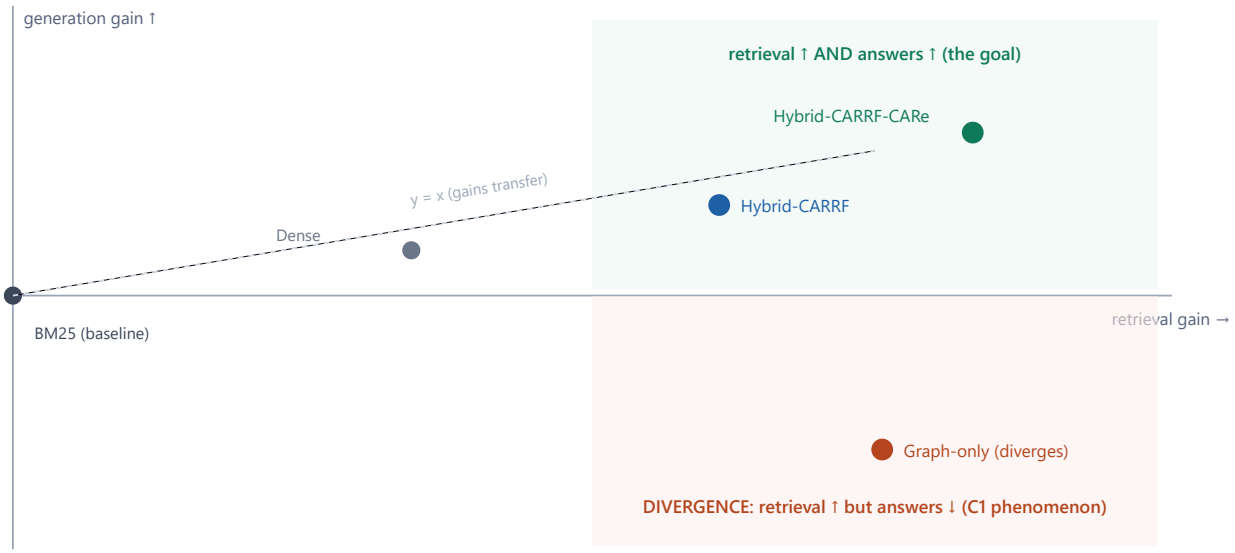

Figure 4. The RGD plane (figure F3 in the paper). The baseline sits at the origin; each system is a point of (retrieval gain, generation gain). Points on the green half delivered answer gains; points in the red lower-right are **divergent** — they retrieved better but answered no better. `diverges` formalizes exactly that quadrant. The thesis' graph system lands there; the contribution arms are designed to climb back onto the diagonal where gains transfer.

Why this is a contribution and not just a plot

Naming and operationalizing the divergence converts a vague worry into a falsifiable claim that travels across benchmarks and backbones. The three-model 'zoo' runs (Table 5) exist precisely to show the phenomenon is not a quirk of one 70B model. RGD is the lens through which the value of C2 and C3 is read: do they move systems out of the divergent quadrant?

4.2 C2 · UMLS grounding & Concept-Aware RRF

C2 has two halves. The first is **grounding**: linking entity mentions to UMLS concept ids (CUIs). The thesis linked only ~47% of entity nodes by exact match; `mgr/graph/umls.py` adds abbreviation expansion and fuzzy matching, and — critically — reports coverage as a **measured curve** rather than a single number, so the value of each tier is visible.

`mgr/graph/umls.py — three linking tiers, each recorded by method`

```
def link_term(self, term: str) -> LinkResult:
    t = normalize_term(term)
    if t in self._exact:            return LinkResult(term, self._exact[t], "exact")
    if t in self._abbrev:           # an abbrev may map to an expansion in the dict
        cui = self._abbrev[t]
        return LinkResult(term, self._exact.get(normalize_term(cui), cui), "abbrev")
    if self.fuzzy_enabled and self._keys:
        match = difflib.get_close_matches(t, self._keys, n=1, cutoff=self.fuzzy_threshold)
        if match:                   return LinkResult(term, self._exact[match[0]], "fuzzy")
    return LinkResult(term, None, "none")
```

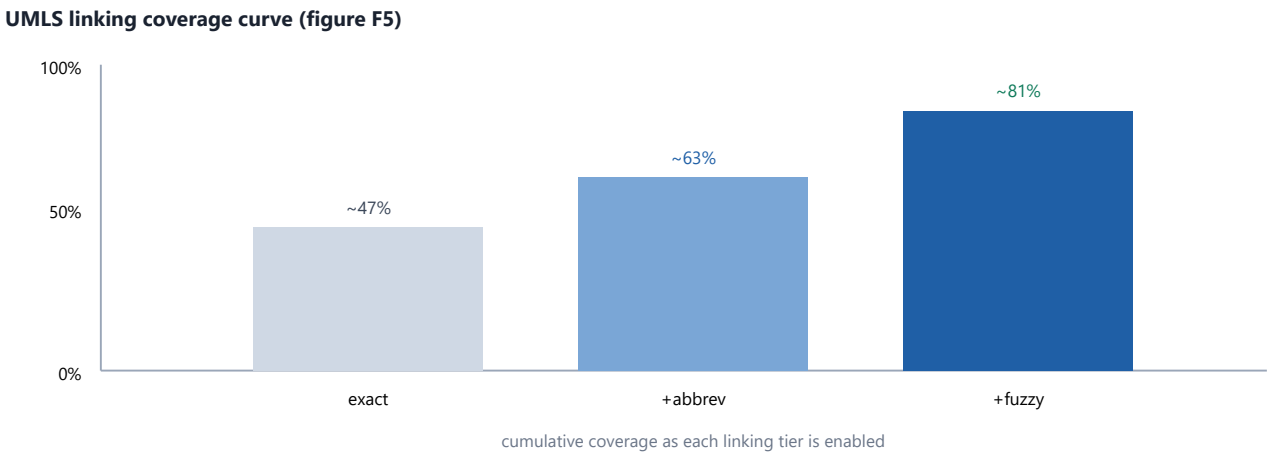

Figure 5. The coverage curve (`CoverageReport.curve`). Each tier is additive: exact match recovers ~47% (the thesis ceiling), abbreviation expansion and fuzzy matching recover the rest. Reporting the curve — not just the final number — is the countermeasure to defect D-6: a reviewer can see precisely how much each mechanism contributes.

The second half is the **fusion**. Reciprocal Rank Fusion (RRF) is the field default: each ranked list contributes `w / (k + rank)` to every document it contains. CA-RRF's idea is to build *one extra ranked list* — candidates ordered by how much their UMLS concept set overlaps the query's concepts — and fuse it with the lexical/dense lists using the **same constant k**. The concept overlap becomes a first-class ranking signal instead of a tunable knob.

`mgr/retrieval/ca_rrf.py — the concept list is just one more RRF input`

```
def ca_rrf(component_lists, query_concepts, candidate_concepts, *,
           k=60, weights=None, concept_metric="count", use_concept=True):
    names = list(component_lists.keys())
    lists = [component_lists[n] for n in names]
    ws    = [ (weights or {}).get(n, 1.0) for n in names ]
    if use_concept:
        clist = concept_ranked_list(query_concepts, candidate_concepts, metric=concept_metric)
        lists.append(clist)                    # the extra concept-overlap list
        ws.append((weights or {}).get("concept", 1.0))
    return reciprocal_rank_fusion(lists, k=k, weights=ws)
```

Isolability: the ablation is built into the API (kills D-4)

Setting `use_concept=False` makes `ca_rrf` compute *exactly* plain RRF over the same component lists, with the same `k` and weights, everything else frozen. So the ablation '{lexical+dense} vs {+concept list}' measures only the concept signal's marginal value — a reviewer cannot dismiss it as a free parameter. The `concept_ranked_list` also `drop_zero`s candidates with no overlap, so the concept list only *boosts* terminology matches rather than re-ordering everything.

4.3 C3 · CARe — cost-aware adaptive reranking

C3 is the headline arm. Cross-encoder reranking improves answers but is expensive — O(c) forward passes per query. CARe is a per-query gate `g(q) ∈ {0,1}` that predicts whether reranking will help, so compute is spent only on the ambiguous queries that need it. The features are **cheap and pre-generation**: they read only the fused candidate scores and concept overlaps, never the model's answer.

Table 8. The six CARe features (`FEATURE_NAMES`). All are computable from the fused candidate list before any generation, so the gate costs almost nothing to evaluate.

Feature	What it captures	Intuition
`top1_gap`	$s_1 - s_2$ (max-normalized)	A clear winner needs no rerank
`top1_topk_gap`	$s_1 - s_k$	Flat top-k = ambiguous
`score_std`	spread of top-k scores	Low spread = hard to separate
`near_tie_frac`	fraction within tol of the top	Many ties = reranking can reorder
`overlap_entropy`	normalized entropy of concept mass	Diffuse concept evidence = uncertain
`query_type`	external signal (e.g. multi-hop)	Some query classes always benefit

The gate is a small standardized logistic regression trained on the **oracle benefit** label — did reranking actually improve that query's answer, computed from results that ran reranking both on and off. At deployment, the decision rule is *cost-aware*: rerank iff the expected quality gain beats its cost.

`mgr/rerank/care_gate.py — a cost-aware decision over a logistic gate`

```
def decide(self, f: CareFeatures, *, value=1.0, cost=0.0) -> bool:
    """Rerank iff expected gain beats cost.
    cost=0 -> a plain probability threshold; cost>0 -> p*value > cost."""
    p = self.predict_proba(f)
    if cost > 0.0:
        return p * value > cost
    return p >= self.threshold

def oracle_benefit(quality_with_rerank, quality_without, *, eps=0.0) -> int:
    """Oracle label: 1 iff reranking strictly improved answer quality."""
    return int(quality_with_rerank - quality_without > eps)
```

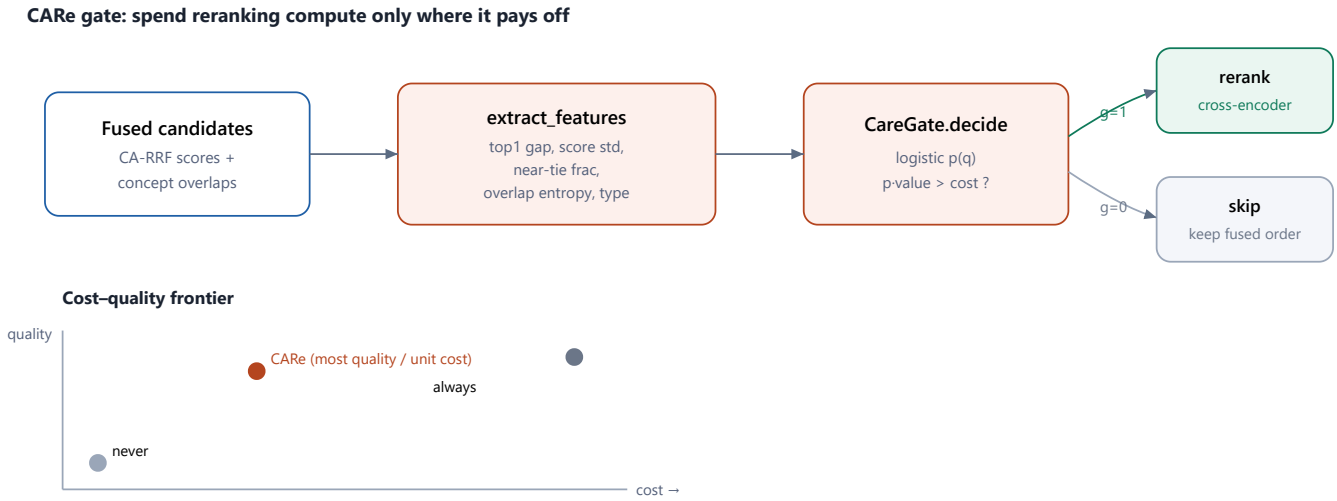

Figure 6. CARe at inference (top) and the evidence it produces (bottom). Cheap features are extracted from the fused list; the logistic gate decides rerank vs skip under a cost rule. The `cost_quality_frontier` helper then compares CARe against always-rerank and never-rerank, yielding the headline claim: CARe recovers most of the quality of always-reranking at a fraction of its cost (it pays only `E[g]·c`).

The `cost_quality_frontier` function turns the gate into the paper's money figure: for each policy it computes mean quality (with-rerank value where the policy fires, no-rerank value otherwise) and total cost (rerank cost per fired query). CARe's cost is `E[g]·c` rather than the full `c` of an always-on reranker.

4.4 The FusionRetriever: one class, every hybrid arm

The three contributions converge in a single class. `FusionRetriever` runs each component retriever, fuses their lists with CA-RRF (concept list on/off), and optionally reranks the top window — never, always, or only when the CARe gate fires. Every hybrid condition in the manifest is just this class with two flags flipped, which is why the executor and prompt path never change across arms.

Table 9. `HYBRID_SPECS`: the mapping from condition name to `(use_concept, rerank_mode)`. One class, two switches, every hybrid arm and ablation.

Condition	use_concept	rerank_mode	What it isolates
Hybrid-RRF2 / RRF4	False	off	Plain field-default fusion
Hybrid-CARRF	True	off	C2: the concept list's value
Hybrid-CARRF-staticRerank	True	static	Always-on rerank (the cost ceiling)
Hybrid-CARRF-CARe	True	adaptive	C3: the gate (HEADLINE)
Hybrid-CARRF-noGrounding	False	off	Grounding ablation
Hybrid-CARRF-noVecIndex	True	off	Flat vs ANN dense index

`mgr/retrieval/fusion.py — the one retrieve() that expresses all hybrids`

```
def retrieve(self, query, *, depth_k=10) -> RetrievalResult:
    pool = max(depth_k, self.rerank_window, 20)
    comp_lists = {name: r.retrieve(query, depth_k=pool).retrieved_ids
                  for name, r in self.components.items()}
    qc = set(self.query_concepts_fn(query)) if self.use_concept else set()
    fused = ca_rrf(comp_lists, qc, self.candidate_concepts or {},
                   k=self.k, weights=self.weights,
                   concept_metric=self.concept_metric, use_concept=self.use_concept)
    fused_ids = [d for d, _ in fused]; score_map = dict(fused)
    rerank_fired = self._maybe_rerank(query, fused_ids, score_map, qc, cc)
    if rerank_fired:
        window = self.reranker.rerank(query, fused_ids[:self.rerank_window], self.passages)
        fused_ids = window + fused_ids[self.rerank_window:]
    final = fused_ids[:depth_k]
    return RetrievalResult(context=render_context(final, self.passages, self.max_context_chars),
                           retrieved_ids=final, ranks={d: i+1 for i, d in enumerate(final)},
                           fused_scores=[score_map.get(d, 0.0) for d in final],
                           rerank_fired=rerank_fired)
```

Note `rerank_fired` is returned on the result and recorded per item — so after a sweep you can measure exactly what fraction of queries CARe chose to rerank, which is the rerank-rate axis of the cost–quality frontier. The `__post_init__` validation refuses an adaptive mode without a gate, or a rerank mode without a reranker, so misconfigured arms fail loudly at construction rather than producing quietly-wrong numbers.

PART IV

The evaluation, end to end

This is the longest part by design. Every number that reaches the paper is produced here, and every step is engineered to be defensible. We walk the per-item loop, then each metric family, then the audit that gates them, then the full statistics pipeline — with the code for each.

5.0 Evaluation philosophy & the validity controls

The evaluation is built on a simple separation: **scoring is per item, comparison is cross-arm, and the two never mix.** A single run scores its own items and writes them to `items.jsonl` keyed by `qid`. Only later, in the statistics layer, are two runs joined on those qids and compared. This is what makes paired tests valid — the same items, answered by every system.

Four validity controls sit on top of the raw metrics. They are not optional add-ons; the pipeline is structured so that a headline number cannot be emitted unless they pass.

Table 10. The four validity controls and where they live. Together they enforce the rule that a headline metric must be computed under an audited identical schema and reported with an honest p and an effect size.

Control	Defends against	Mechanism	Module
Prompt parity	Wording differences masquerading as condition effects	One frozen, hashed prompt set across all arms	`generate/prompts.py`
Answer-format audit	The EM 0.00→0.76 schema artifact (D-1)	Shared normalizer + cross-arm schema/coverage gate before any EM/F1	`eval/answer_format_audit.py`
Embedding parity	'Embedding named two ways' (D-7)	One frozen `embedding_checkpoint`, asserted into every record	`configs/base.yaml`
Honest significance	The 1/N permutation floor (D-2)	≥100k permutations; refuses to report the floor as a point p	`stats/permutation.py`

5.1 The generate → extract → score loop

`RAGExecutor` is the real per-item engine that the runner drives. It is identical for every condition; the only injected variable is the retriever (a `NullRetriever` for No-RAG, BM25/dense/graph/fusion otherwise). For each item it: retrieves context, builds the shared prompt, generates, normalizes the answer, and scores it — appending a fully-populated per-item record.

`mgr/generate/executor.py — the per-item record is the substrate for everything downstream`

```
for it in items:
    rr = RetrievalResult()
    try:
        rr = self.retriever.retrieve(it.question, depth_k=depth_k)
        msgs = prompts.build_messages(it.question, it.answer_type,
                                      options=it.options, context=rr.context)
        t0 = time.time()
        raw, usage = self.client.complete_text(model_id, msgs, **decoding)
        latency = time.time() - t0
    except Exception as e:          # one bad item must not sink the whole run
        error = f"{type(e).__name__}: {e}"
    norm = normalize(raw, it.answer_type)
    correct = exact_match(norm, it.gold)
    out_items.append({
        "qid": it.qid, "retrieved_ids": rr.retrieved_ids, "ranks": rr.ranks,
        "rerank_fired": rr.rerank_fired, "answer_raw": raw, "answer_norm": norm,
        "gold": it.gold, "correct": bool(correct), "em": correct,
        "f1": token_f1(norm, it.gold),
        "tokens": {"in": ..., "out": ...}, "latency_s": latency,
    })
```

Three deliberate choices in this loop

- **Per-item isolation.** A single failing item is caught and recorded as an error rather than crashing the whole run, so a 1,089-item benchmark is not lost to one malformed question.
- **Deterministic decoding.** `decoding["seed"] = row.seed` — the decoding seed comes from the run's seed dimension, so a run is reproducible bit-for-bit given the same backbone.
- **Everything is recorded.** retrieved ids, ranks, whether rerank fired, raw *and* normalized answers, gold, EM, F1, tokens, latency — so any downstream metric or audit can be recomputed without re-running generation.

The executor also computes `expected_n_items` — the smoke subset size if set, else the benchmark's declared count — and hands it to the runner's integrity check. That is how a truncated data load is caught on a full sweep while a deliberate 200-item smoke run still passes.

5.2 Prompt parity: frozen, hashed, identical across arms

The prompt set is versioned and hashed; the hash lands in every run-record. There is one system message and one terse output-format instruction per answer type. Keeping these identical across arms is what makes the normalized output schema constant — the precondition for the audit.

`mgr/generate/prompts.py — the format instruction is part of the hashed contract`

```
FORMAT_INSTRUCTION = {
    "mcq":          "Respond with a single capital letter (A, B, C, or D) and nothing else.",
    "yes_no_maybe": "Respond with exactly one word: yes, no, or maybe.",
    "yes_no":       "Respond with exactly one word: yes or no.",
    "free":         "Answer concisely and ground every claim in the provided context.",
}

def prompt_set_hash() -> str:
    blob = json.dumps({"version": PROMPT_SET_VERSION, "system": SYSTEM,
                       "format": FORMAT_INSTRUCTION}, sort_keys=True, ensure_ascii=False)
    return hashlib.sha256(blob.encode("utf-8")).hexdigest()
```

`build_messages` assembles the chat turns in a fixed order — optional context block, the question, options, then the format instruction — with the retrieved context being the *only* difference between a RAG arm and No-RAG. The answer-type taxonomy (mcq / yes-no-maybe / yes-no / free) is derived from the Benchmarks sheet `type` column, so each benchmark automatically gets the right instruction and the right normalizer.

5.3 Answer extraction & normalization

A raw completion is mapped to a canonical label by `normalize`. The same normalizer is applied to **every** arm — this is half of the cure for the EM artifact. It returns `None` on an unextractable answer (an explicit miss, never a silent default), which is what makes coverage measurable.

`mgr/generate/extract.py — deterministic, returns None on a true miss`

```
def _extract_mcq(text: str) -> str | None:
    m = re.search(r"\b([ABCD])\b", text.upper())          # lone letter: "B", "B.", "Answer: B"
    if m: return m.group(1)
    m = re.search(r"[\(\[]?([ABCD])[\)\].:]", text.upper())  # "(B)" / "B)" fallback
    return m.group(1) if m else None

def _extract_yesno(text, allow_maybe):
    m = re.search(r"\b(yes|no|maybe)\b", text.lower())
    if not m: return None
    label = m.group(1)
    return None if (label == "maybe" and not allow_maybe) else label
```

Two companions support the audit: `label_schema` reports the realized label vocabulary an arm actually produced (excluding misses), and `expected_schema` returns the closed label set a typed benchmark *should* produce (`{A,B,C,D}`, `{yes,no,maybe}`, …) or `None` for free-form. The gap between the two is precisely what the audit inspects.

5.4 Generation metrics: EM, token-F1, accuracy, coverage

Generation metrics consume the normalized predictions. Exact match is a normalized string equality; token-F1 is the standard overlap F1 over alphanumeric tokens with proper multiset counting; accuracy is mean EM over labelled items (the MCQ/yes-no headline). The fourth metric, **coverage**, is the fraction of items with an extractable prediction — a number that only exists because the normalizer is honest about misses.

`mgr/generate ... metrics/generation.py — EM/F1/accuracy/coverage`

```
def token_f1(pred, gold) -> float:
    p, g = _tokens(pred), _tokens(gold)
    common, gset = {}, list(g)
    for t in p:                                  # multiset intersection
        if t in gset:
            common[t] = common.get(t, 0) + 1; gset.remove(t)
    n_common = sum(common.values())
    if n_common == 0: return 0.0
    precision = n_common / len(p); recall = n_common / len(g)
    return 2 * precision * recall / (precision + recall)

@dataclass
class GenerationScores:
    n: int; accuracy: float; em: float; f1: float
    coverage: float       # fraction of items with an extractable prediction
```

EM/F1 are not reported until the audit says so

The docstring of this module is explicit: *'EM/F1 must only be reported after the answer-format audit passes'*. The metrics functions will happily compute numbers, but the stats layer refuses to surface them for a comparison family that fails the audit — it surfaces the audit report instead. Coverage is the early-warning signal: a sudden drop means an arm's parser degenerated.

5.5 The answer-format audit — the centerpiece control

This is the single most important validity control in the repository, because it directly kills the thesis' most embarrassing artifact: EM jumping 0.00→0.76 for hybrid arms only, under the *same* generator. That happened because arms were scored under different normalized output schemas. The audit runs **first**, over a family of arms scored on the same items, and refuses to emit EM/F1 if either condition fails.

- **Conformance.** Every arm's realized labels must be a subset of the expected closed schema for the answer type (for typed benchmarks; free-form has no schema gate).
- **Coverage floor.** No arm's extraction coverage may fall below a floor (default 0.95) — a degenerate parser is caught rather than silently scored as wrong.

`mgr/eval/answer_format_audit.py — gates EM/F1 across a comparison family`

```
def audit(arm_labels, answer_type, *, min_coverage=0.95) -> AuditReport:
    report = AuditReport(answer_type=answer_type, passed=True)
    exp = expected_schema(answer_type)            # None for free-form
    for arm, labels in arm_labels.items():
        schema = label_schema(labels, answer_type)
        coverage = sum(1 for x in labels if x is not None) / len(labels)
        conforms = True if exp is None else schema.issubset(exp)
        report.arms.append(ArmAudit(arm, schema, coverage, conforms))
        if exp is not None and not conforms:
            report.passed = False
            report.reasons.append(f"{arm}: labels {sorted(schema - exp)} outside expected {sorted(exp)}")
        if coverage < min_coverage:
            report.passed = False
            report.reasons.append(f"{arm}: coverage {coverage:.3f} < floor {min_coverage}")
    return report
```

The subtlety that makes the audit correct

The audit gates on **conformance + coverage**, not on realized-set *equality* across arms. If arm A happens to answer only 'A/B' and arm B answers 'A/B/C/D', that is a difference in the *answer distribution*, not a schema mismatch — both are valid under one shared normalizer and one expected vocabulary. The original artifact was a genuine *schema* divergence (different normalizers), which this design makes structurally impossible: one normalizer, one expected closed set, every arm scored the same way.

`gate_emit_em_f1()` on the report is the boolean the stats layer checks. When it is `False`, the divergence becomes a *stated finding* — the audit report is reported — rather than a silent, impressive-looking number.

5.6 Retrieval metrics: Recall@k, Precision@k, MRR, nDCG

Retrieval metrics score a ranked id list against a set of gold ids per query, then aggregate over a benchmark. **Recall@3** is the sheet's primary retrieval metric; MRR and nDCG@10 are also flagged primary. Together with the generation metrics they are the two coordinates of every RGD point — retrieval gain on one axis, generation gain on the other.

`mgr/metrics/retrieval.py — the standard ranking metrics, carefully normalized`

```
def recall_at_k(retrieved, relevant, k) -> float:
    rel = set(relevant)
    if not rel: return 0.0
    hit = sum(1 for d in retrieved[:k] if d in rel)
    return hit / len(rel)

def reciprocal_rank(retrieved, relevant) -> float:
    rel = set(relevant)
    for i, d in enumerate(retrieved, start=1):
        if d in rel: return 1.0 / i
    return 0.0

def ndcg_at_k(retrieved, relevant, k) -> float:        # binary-gain nDCG
    dcg  = sum(1.0/math.log2(i+1) for i, d in enumerate(retrieved[:k], 1) if d in set(relevant))
    idcg = sum(1.0/math.log2(i+1) for i in range(1, min(len(set(relevant)), k)+1))
    return dcg/idcg if idcg > 0 else 0.0
```

The aggregator `score()` sweeps the standard k-sets (`Recall@{1,3,5,10}`, `Precision@{1,3,5}`, `nDCG@10`) and means them over the benchmark, returning a tidy `RetrievalScores`. These metrics apply only to the benchmarks whose `metric_focus` is *Retrieval+Gen* (PubMedQA, BioASQ-YN, the long-form sets) — an MCQ exam has no gold passage to recall.

5.7 Grounding: real RAGAS with an injected judge

The thesis used a proxy for 'grounding'; the journal upgrade demands the real thing. `grounding_ragas.py` implements four RAGAS metrics — faithfulness, answer relevance, context precision, context recall — each an LLM-judged quantity. The judge is an **injected protocol**: the NIM judge at runtime, a deterministic fake in tests. This is what lets the grounding logic be unit-tested without a model.

`mgr/metrics/grounding_ragas.py — judge is a Protocol, so grounding is testable offline`

```
class GroundingJudge(Protocol):
    def decompose(self, text) -> list[str]: ...        # answer/reference -> atomic claims
    def entails(self, hypothesis, premise) -> bool: ... # does context support a claim?
    def relevant(self, question, passage) -> bool: ...  # is a passage relevant?
    def relevance(self, question, answer) -> float: ... # how relevant is the answer? [0,1]

def faithfulness(answer, contexts, judge) -> float:
    """Fraction of answer claims supported by the retrieved context."""
    claims = judge.decompose(answer)
    if not claims: return 1.0                  # nothing asserted -> nothing unsupported
    premise = "\n".join(contexts)
    return sum(1 for c in claims if judge.entails(c, premise)) / len(claims)

def context_precision(question, contexts, judge) -> float:
    rels = [judge.relevant(question, c) for c in contexts]
    n_rel = sum(rels)
    if n_rel == 0: return 0.0
    num, hits = 0.0, 0
    for k, r in enumerate(rels, start=1):
        if r: hits += 1; num += hits / k       # mean precision@k over relevant ranks
    return num / n_rel
```

Table 11. The four RAGAS grounding metrics. Answer relevance is reported beside token-F1 as a first-class finding, controlling for the brevity/relevance trade-off the thesis missed.

RAGAS metric	Question it answers	How it is computed
Faithfulness	Is the answer supported by the retrieved context?	Claims entailed by context / total answer claims
Answer relevance	Does the answer address the question?	Judge relevance score, clamped to [0,1]
Context precision	Is the retrieved context on-topic?	Mean precision@k over relevant-context ranks
Context recall	Does the context cover the needed evidence?	Reference claims entailed by context / total

Why injecting the judge matters

By depending only on the `GroundingJudge` protocol, the four metrics are pure functions of (judge decisions, texts). In tests a fake judge with known behavior pins the arithmetic exactly; at runtime the NIM judge (a permitted, low-volume free-tier use) supplies real decisions. The grounding budget — ~70M judge tokens — is accounted separately from the generation envelope precisely because it is post-hoc and on a free tier.

5.8 Cost metering: estimate vs actual

Two meters coexist. The **estimate** reproduces the manifest's `est_tokens / est_cost` exactly (the $591/$887 envelope). The **actual** tracks observed tokens at the backbone's blended rate. Because RunPod bills by GPU-hour, the per-token actual is reported alongside a GPU-hours actual so the budget reconciles whichever way you slice it.

`mgr/metrics/cost.py — estimate and actual, both on every record`

```
@dataclass(frozen=True)
class CostMeter:
    est_tokens: float; est_cost_usd: float
    actual_tokens: int = 0; actual_cost_usd: float = 0.0

    @classmethod
    def from_estimate(cls, n_items, tokens_per_item, ctx_factor, rate_per_m):
        et = est_tokens(n_items, tokens_per_item, ctx_factor)   # n * tok * ctx
        return cls(est_tokens=et, est_cost_usd=et / 1e6 * rate_per_m)

    def with_actuals(self, tokens_total, rate_per_m):          # folded in post-run
        return CostMeter(self.est_tokens, self.est_cost_usd,
                         actual_tokens=tokens_total,
                         actual_cost_usd=tokens_total / 1e6 * rate_per_m)
```

5.9 The statistics layer

This is where raw per-item records become defensible claims. The pipeline is a fixed sequence: align two runs by qid → bootstrap a confidence interval → run a paired permutation test for significance → correct across the comparison family with Holm–Bonferroni → report an effect size beside every p. Each stage is its own small, tested module.

5.9.1 Pairing: align two runs by qid

Paired tests require identical item sets. `align_by_qid` joins two runs on their `qid`s and — importantly — **raises** if the sets differ, surfacing a real defect rather than silently comparing different items.

`mgr/stats/io.py — refuses to mis-pair; identical item sets or an error`

```
def align_by_qid(items_a, items_b, metric="em"):
    a = {str(it["qid"]): it for it in items_a}
    b = {str(it["qid"]): it for it in items_b}
    shared = sorted(set(a) & set(b))
    if not shared: raise ValueError("no shared qids between the two runs")
    miss_a, miss_b = set(b) - set(a), set(a) - set(b)
    if miss_a or miss_b:
        raise ValueError(f"qid sets differ: {len(miss_b)} only in A, {len(miss_a)} only in B")
    va = np.array([float(a[q][metric]) for q in shared])
    vb = np.array([float(b[q][metric]) for q in shared])
    return va, vb, shared
```

5.9.2 Bootstrap confidence intervals

Every headline number gets a 95% CI from 10,000 percentile-bootstrap resamples, deterministic given a seed. The paired variant resamples the per-item differences together, which is the right interval for a same-items comparison.

`mgr/stats/bootstrap.py — 10k percentile resamples, paired on differences`

```
def bootstrap_ci(values, *, n_boot=10_000, level=0.95, seed=0) -> CI:
    rng = np.random.default_rng(seed)
    idx = rng.integers(0, len(values), size=(n_boot, len(values)))
    means = values[idx].mean(axis=1)                # vectorized resampling
    lo, hi = np.quantile(means, [(1-level)/2, 1-(1-level)/2])
    return CI(point=float(values.mean()), lo=float(lo), hi=float(hi), n_boot=n_boot, level=level)

def bootstrap_diff_ci(a, b, **kw) -> CI:            # paired: CI of mean(a - b)
    return bootstrap_ci(a - b, **kw)
```

5.9.3 Paired permutation test — and the refusal to report the floor

This module is a direct fix for defect D-2. A two-sided paired sign-flip permutation test on `mean(a−b)`, with ≥100k permutations. When the sample is tiny (≤20 items) it enumerates the full sign-flip space for an *exact* p. The crucial behavior: when the observed statistic is never exceeded, it does **not** report `p ≈ 1/N` as if it were a point estimate — it flags `at_floor` and offers `assert_resolved()` to make the pipeline raise.

`mgr/stats/permutation.py — exact when small, honest about the floor when large`

```
def paired_permutation_test(a, b, *, n_perm=100_000, seed=0, tol=1e-12) -> PermResult:
    d = a - b; n = len(d); obs = float(d.mean()); abs_obs = abs(obs)
    if n <= 20 and (2**n) <= n_perm:                # exact enumeration when feasible
        signs = _all_sign_vectors(n)
        stats = (signs * d).mean(axis=1)
        count = int(np.sum(np.abs(stats) >= abs_obs - tol))
        return PermResult(obs, count/signs.shape[0], signs.shape[0], exact=True, at_floor=False)
    rng = np.random.default_rng(seed); exceed = 0; done = 0
    while done < n_perm:                             # streamed in blocks to bound memory
        m = min(block, n_perm - done)
        signs = rng.choice(np.array([-1.0, 1.0]), size=(m, n))
        exceed += int(np.sum(np.abs((signs*d).mean(axis=1)) >= abs_obs - tol)); done += m
    at_floor = exceed == 0
    p = (exceed + 1) / (n_perm + 1)                 # add-one smoothing; never exactly 0
    return PermResult(obs, p, n_perm, exact=False, at_floor=at_floor)

def assert_resolved(self):
    if self.at_floor:
        raise ValueError("permutation p hit the resolution floor; "
                         "increase n_perm or report the bound, never the floor as a point estimate")
    return self
```

The thesis number that this single function retires

The original study reported an *identical* p = 0.00025 across three different comparisons. 0.00025 ≈ 1/4000 is exactly the resolution floor of a 4,000-permutation test — not a measured probability but the smallest number the test could emit. Here, hitting that floor is a `at_floor=True` flag and an optional hard error, forcing the analyst to either raise the permutation count or report an honest bound `p < 1/(N+1)`.

5.9.4 Holm–Bonferroni & effect sizes

Comparing many arms inflates family-wise error, so raw p-values are corrected with step-down Holm–Bonferroni — uniformly more powerful than plain Bonferroni while still controlling family-wise error. And because significance without magnitude is misleading, every p is reported beside a paired Cohen's d (difference-scale magnitude) and Cliff's delta (a non-parametric dominance measure robust to the binary EM/correct metrics common here).

`mgr/stats/holm.py + effect_size.py — correction and magnitude, together`

```
def holm_bonferroni(pvalues, *, alpha=0.05) -> list[HolmResult]:
    ordered = sorted(pvalues.items(), key=lambda kv: kv[1]); prev_adj = 0.0
    still_rejecting = True; results = []
    for i, (label, pv) in enumerate(ordered):
        adj = max(min(1.0, (len(pvalues) - i) * pv), prev_adj)   # clamp + monotone
        prev_adj = adj
        reject = still_rejecting and pv <= alpha / (len(pvalues) - i)
        if not reject: still_rejecting = False
        results.append(HolmResult(label, pv, adj, reject))
    return results

def cliffs_delta(a, b) -> float:                    # P(a>b) - P(a<b) over all pairs
    diff = a[:, None] - b[None, :]
    return float((np.sum(diff > 0) - np.sum(diff < 0)) / (len(a) * len(b)))
```

5.10 The metric catalog & the statistical protocol

Pulling it together, the workbook's Metrics sheet declares twenty metrics across four families, flagging which are *primary* (always reported with a CI). The Statistics sheet pins the protocol the stats layer implements.

Table 12. The 20-metric catalog (Metrics sheet). Primary metrics (bold) are the ones that always carry a bootstrap CI and enter the Holm family; Human-Faithfulness anchors the RAGAS judge against expert ratings.

Family	Metrics (primary in bold)	Applies to
Retrieval	Recall@1/5/10, Precision@3, **Recall@3**, **MRR**, **nDCG@10**	Retrieval+Gen benchmarks
Generation	**Accuracy**, **Exact Match**, **Token-F1**, BERTScore-F1	MCQ / yes-no / long-form
Grounding	**RAGAS-Faithfulness**, AnswerRelevance, ContextPrecision/Recall, **Human-Faithfulness**	Free-text subset / human-rated
Cost	Tokens/query, Latency-p50, **Latency-p95**, **USD/1k queries**	All conditions

Table 13. The statistical protocol (Statistics sheet) — a one-to-one specification of the `mgr/stats` modules walked above.

Protocol parameter	Value	Rationale
Seeds (headline)	42, 123, 7	Three runs for variance on every main comparison
Seeds (ablation / zoo)	42	One seed; extend to 3 if a delta sits near its CI
Bootstrap resamples	10,000	Per-metric 95% confidence intervals
Significance test	Paired permutation (item-level)	Same items answered by all systems
Permutations	≥ 100,000	Avoid the 1/N floor; report exact p, never a floor
Multiple-comparison correction	Holm–Bonferroni	Controls family-wise error across arms
Effect size	Paired Cohen's d / Cliff's delta	Reported alongside every p-value
Alpha	0.05 (two-sided)	—
Answer-format audit	Mandatory before any F1/EM claim	Resolves the EM 0.00→0.76 anomaly

PART V

Serving, data & lifecycle

The harness is offline-complete, but the sweep needs a GPU. This part covers the serving clients, the three-layer safety net that guarantees a paid pod never lingers, the data pipeline, the test suite, and the end-to-end runbook that ties everything together.

6 Clients, serving & the GPU-pod safety net

There are two serving roles, and the code structurally prevents mixing them up. Embeddings, the reranker, the RAGAS judge, and graph triple-extraction run on the **NIM free tier**, throttled to ≤40 req/min. Bulk 70B **generation** runs on a **local vLLM** server on the A100 — never on NIM, because free-tier throttling would cripple the sweep. The NIM client enforces this with a hard guard: calling generation on it raises.

`mgr/clients/nim.py — the free tier physically refuses to route generation`

```
class NimClient:
    def embeddings(self, model, inputs, **p): return self._client.embeddings(model, inputs, **p)
    def judge(self, model, messages, **p):     return self._client.chat(model, messages, **p)  # RAGAS ok
    def chat(self, *a, **k):                                        # generation guard
        raise GenerationOnNIMError(
            "generation must run on local vLLM (A100), not the NIM free tier; "
            "use mgr.clients.vllm.VLLMClient")
```

Both clients share a rate-limited, retry-backed OpenAI-compatible core (`openai_compat.py`) with an injectable transport, clock, and sleep — so the token-bucket limiter and exponential-backoff-with-jitter retry are unit-testable without a network or real time. The vLLM client leaves the rpm cap effectively unbounded because the binding resource there is GPU saturation via continuous batching, not request rate.

6.1 The three-layer pod safety net

A forgotten GPU pod is the most expensive mistake in cloud research. Three independent layers protect against it, so any single failure is caught by another:

Table 14. The defense-in-depth that bounds GPU spend. The runner touches a heartbeat file; the watchdog — running on a cheap always-on host, never the GPU — tears the pod down if the heartbeat ages past the threshold.

Layer	Mechanism	Triggers on
1 · Session trap	`with_pod.sh` traps EXIT/crash/Ctrl-C and tears the pod down	The wrapping process ending, any way
2 · Idempotent down	`down.sh` can be run any number of times safely	Manual or automated teardown
3 · Idle watchdog	`pod_watch.py` kills a pod whose heartbeat went stale	No active run past the idle threshold

`infra/guards/pod_watch.py — the idle decision is pure and unit-tested`

```
def is_idle(last_heartbeat_ts, now, idle_threshold_s) -> bool:
    return (now - last_heartbeat_ts) > idle_threshold_s

def check_once(heartbeat, down_script, *, idle_threshold_s, now=None) -> str:
    hb = read_heartbeat(heartbeat)              # file mtime, or None
    if hb is None:                  return "no-heartbeat-yet"
    if is_idle(hb, now or time.time(), idle_threshold_s):
        teardown(down_script);      return "torn-down"
    return "alive"
```

7 The data pipeline: MIRAGE → JSONL

QA data is small (tens of MB) and lives locally; the heavy corpora and embeddings are fetched separately onto the cloud volume. `convert_mirage.py` maps MIRAGE's single `benchmark.json` into one normalized JSONL per manifest benchmark; the loader reads that one schema for both the 200-item smoke and the full sweep, so the data path never forks.

`mgr/data/convert_mirage.py + loader.py — one normalized schema, smoke == sweep`

```
BENCHMARK_TO_MIRAGE = {"MMLU-Med": "mmlu", "MedQA-US": "medqa", "MedMCQA": "medmcqa",
                       "PubMedQA": "pubmedqa", "BioASQ-YN": "bioasq"}

def load_items(benchmark, data_root, *, benchmark_type="MCQ", n_items=None):
    atype = answer_type_for(benchmark_type)      # sheet 'type' -> internal answer type
    items = []
    for line in open(benchmark_path(benchmark, data_root)):
        d = json.loads(line)
        items.append(BenchmarkItem(qid=str(d["qid"]), question=str(d["question"]),
                     gold=(None if d.get("answer") is None else str(d["answer"])),
                     answer_type=atype, options=d.get("options")))
        if n_items is not None and len(items) >= n_items: break
    return items
```

8 Testing & verification (118 tests)

The headline claim — *everything buildable without a GPU, a judge, or real data is done and tested* — is backed by 118 offline tests. Every contribution has tested core logic; every validity control has a test that proves it fires. The table is the test census.

Table 15. The 118-test census (selected). The largest suites guard the three contributions and the validity controls — exactly where a reviewer's scrutiny will land.

Test file	n	What it proves
`test_generation.py`	10	EM/F1/accuracy/coverage arithmetic
`test_care.py`	9	**C3** features, logistic fit, cost rule, frontier
`test_fusion.py`	9	RRF + **C2** CA-RRF, isolable ablation
`test_stats.py`	9	**C1** RGD, CIs, permutation floor, Holm, effect sizes
`test_manifest.py`	8	Validation invariants + cost reconciliation
`test_fusion_retriever.py`	7	Every hybrid arm through one tested path
`test_grounding.py`	7	RAGAS metrics under a fake judge
`test_umls.py`	7	Exact/abbrev/fuzzy linking + coverage curve
`test_runner.py`	7	Claim/record/resume/integrity lifecycle
`test_ids / state / clients / dense / retrieval / retrieval_metrics`	6 each	Identity, state machine, rate limiter, dense index, metrics
`test_pod_watch.py`	5	The idle decision and teardown trigger
`test_convert_mirage / executor / smoke`	2 each	Data conversion, the RAGExecutor, the H2 smoke

9 From clean repo to the paper: the runbook

The end-to-end procedure is an ordered, gate-driven checklist. The shape matters as much as the steps: **all the free, offline work happens first**, the GPU only appears once the harness has been smoke-validated, and each phase flips exactly one gate that unblocks the next wave of runs.

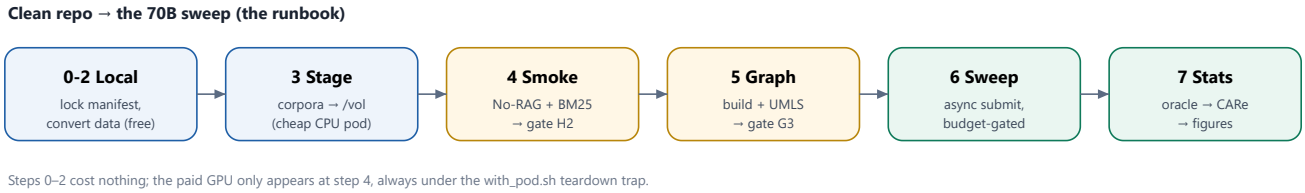

Figure 7. The runbook as a pipeline. Local freeze and data conversion (blue) cost nothing; the smoke and graph build (amber) each set a gate; the paid sweep and the statistics/figures (green) follow. The budget gate halts the sweep if projected actual cost exceeds the ceiling, and the idle watchdog runs throughout.

A `PASS` from the smoke means: a deterministic run, ids matching the manifest, both arms processing the intended 200 items, and metrics emitted. That evidence is what justifies flipping `gates.H2.satisfied: true` — and from there the program unrolls gate by gate to the locked statistics and the final figures (RGD plane F3, UMLS coverage F5, the cost–quality frontier).

Current state of the program

The offline pipeline is **complete and tested (118 tests)**: the harness, all retrieval arms, UMLS grounding, the CARe gate, every metric family, the audit, and the full statistics layer. What remains is the data/cloud phase — the Neo4j graph build and traversal retriever (gate G3), the NIM judge adapter at runtime, and the paid 70B sweep on the A100 — all of which slot into the same tested interfaces walked in this document.

A Appendix: glossary & file index

A.1 Glossary

Table 16. Glossary of the recurring terms in this document and the codebase.

Term	Meaning
RGD	Retrieval-Generation Decomposition (C1): retrieval gain vs generation gain per system
CA-RRF	Concept-Aware Reciprocal Rank Fusion (C2): a UMLS-concept-overlap list fused via RRF
CARe	Cost-Aware Adaptive Reranking (C3): a per-query gate that reranks only when it pays off
RRF	Reciprocal Rank Fusion: `Σ w/(k+rank)`; k=60 default
UMLS / CUI	Unified Medical Language System; a Concept Unique Identifier
RAGAS	Retrieval-Augmented Generation Assessment: faithfulness, relevance, context precision/recall
Gate (H2/G3/P3)	A boolean in the ledger; a row is Ready iff its gate is satisfied
Effective config	`base + condition + benchmark + backbone + seed`, hashed to `config_hash`
Prompt parity	Identical, hashed prompt set across all arms — the experimental-design backbone
Oracle benefit	1 iff reranking strictly improved a query's answer; the CARe training label
Coverage	Fraction of items with an extractable normalized prediction
at_floor	Permutation flag: the observed stat was never exceeded; p is a bound, not a point

A.2 Where to read each contribution in the source

Table 17. A reading map for the three contributions and the controls that make them publishable.

Contribution	Core file(s)	Test(s)	Figure
C1 RGD	`mgr/stats/rgd.py`	`test_stats.py`	F3 / Fig 4
C2 CA-RRF	`retrieval/ca_rrf.py`, `graph/umls.py`	`test_fusion.py`, `test_umls.py`	F5 / Fig 5
C3 CARe	`rerank/care_gate.py`	`test_care.py`	Frontier / Fig 6
Hybrid wiring	`retrieval/fusion.py`	`test_fusion_retriever.py`	Fig 1
Validity controls	`eval/answer_format_audit.py`, `stats/permutation.py`	`test_stats.py`	Table 1, 10

One sentence to take away

medgraphrag-journal is an experiment compiler: a spreadsheet of 244 runs is frozen to a hashed contract, executed one crash-safe row at a time through a single prompt-parity loop, and measured by a statistics layer that refuses every shortcut the original thesis took — so that the three contributions, RGD, CA-RRF, and CARe, can be claimed with confidence intervals, honest p-values, and effect sizes a reviewer will accept.

Generated as a read-alongside course for the medgraphrag-journal repository · all code excerpts are verbatim from the source tree at the documented commit · 2026-06-27.